

LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Branch: CSM

Section: B

Task No.: Assignment 2

Student Name: M. Niharika

Roll No.: A24126552097

1. TITLE

Build a Student Record Management System using MongoDB

2. AIM

To develop a simple **Student Record Management System** using MongoDB that can store, retrieve, update, and manage student information through CRUD operations, filtering, sorting, and indexing.

3. OBJECTIVE

The objectives of this assignment are:

- To create a MongoDB database (collegeDB) and a collection (students) inside it.
- To insert multiple student documents into the collection using insertMany().
- To display all student records using find().
- To filter and display students belonging to a particular branch.
- To filter and display students who scored more than a given number of marks.
- To search for a single student using their rollNo.
- To search students using a different condition, such as their year of study.
- To update one field (marks) and a different field (email) of a specific student using updateOne().
- To delete a student record using rollNo with deleteOne().
- To display students sorted in descending order of marks.
- To create an index on rollNo and understand why indexing speeds up searches.
- To run further real-time queries: students above 80 marks, below 50 marks, the highest scorer, students of a given branch, and students sorted by marks.

4. THEORY

MongoDB is a NoSQL, document-oriented database. Instead of rows and columns, it stores data as *documents* — JSON-like records (technically BSON) — grouped into *collections* inside a *database*. A collection does not enforce a fixed schema, so documents in the same collection can have different fields if needed.

`use("collegeDB")` switches the shell's context to the collegeDB database, creating it the first time data is actually written to it. `db.createCollection("students")` explicitly creates the students collection inside it.

CRUD operations in MongoDB map onto four kinds of methods:

- Create — `insertOne()` / `insertMany()` add one or more documents to a collection.
- Read — `find()` retrieves documents, optionally filtered by a query object, sorted with `.sort()`, and limited with `.limit()`.
- Update — `updateOne()` / `updateMany()` modify fields of matching documents, typically using the `$set` operator to change specific fields without touching the rest of the document.
- Delete — `deleteOne()` / `deleteMany()` remove documents that match a filter.

Query operators such as `$gt` (greater than) and `$lt` (less than) let a filter compare a field's value instead of only matching it exactly, e.g. `{ marks: { $gt: 75 } }` matches every document whose marks field is above 75.

Indexing: without an index, MongoDB must perform a *collection scan* (COLLSCAN) — reading every single document to check whether it matches a query — which gets slower as the collection grows. An index, created with `createIndex()`, builds a separate, sorted data structure (a B-tree) on the indexed field. A query that filters or sorts on an indexed field can then use an *index scan* (IXSCAN) to jump almost directly to the matching documents instead of reading the whole collection, which is why `db.students.createIndex({ rollNo: 1 })` makes searching by rollNo far faster once the students collection grows large — the difference between checking every document and looking the value up directly.

5. HOW TO BUILD THIS PROJECT — STEP-BY-STEP SETUP GUIDE

Install MongoDB Community Server

Download and install MongoDB Community Server for your OS from [mongodb.com](https://www.mongodb.com). On Windows/macOS the installer also offers to install MongoDB Compass (a GUI) alongside it.

Install the MongoDB Shell (mongosh)

mongosh is the modern command-line shell used to run JavaScript-style queries against MongoDB. It is bundled with recent installers, or can be installed separately from the MongoDB Download Center.

Start the MongoDB server (mongod)

Run the mongod process so a database server is listening (by default on port 27017). On Windows this usually runs as a service automatically after installation; on macOS/Linux it may need to be started manually, e.g. `mongod --dbpath /path/to/data`.

Connect using mongosh

Open a terminal and run `mongosh`. This connects to the local MongoDB server and opens an interactive JavaScript-style shell where database commands can be typed directly.

Switch to (create) the database

Run `use("collegeDB")` (or `use collegeDB` in older shell syntax). MongoDB does not create the database file on disk until the first document is actually inserted into it.

Create the students collection

Run `db.createCollection("students")` to explicitly create the collection. (This step is optional in practice — inserting a document into a non-existent collection creates it automatically — but doing it explicitly here documents the step clearly.)

Insert sample student documents

Run `db.students.insertMany([...])` with an array of student objects to populate the collection with starting data.

Run read, update, and delete queries

Use `find()`, `updateOne()`, and `deleteOne()` with the filters and operators described in this report to retrieve, modify, and remove records.

Create an index and verify it

Run `db.students.createIndex({ rollNo: 1 })` and then `db.students.getIndexes()` to confirm the index now exists alongside the default `_id` index.

(Optional) Save the script and re-run it

Save all the commands in a `.js` file (e.g. `mongo_script.js`) and run the whole file at once from a terminal with `mongosh collegeDB mongo_script.js`, instead of typing each command interactively.

(Optional) Inspect the data visually

Open MongoDB Compass and connect to `localhost:27017` to browse the `collegeDB` database and the `students` collection in a table/document view, which is often easier for spot-checking data than reading raw shell output.

6. ALGORITHM / PROCEDURE

- Switch to the collegeDB database with use("collegeDB") and create the students collection.
- Insert four student documents in one call using insertMany(), each with rollNo, name, branch, year, marks, and email.
- Retrieve and display every document in the collection using find().
- Filter and display only the documents where branch equals "CSE-AIML".
- Filter and display only the documents where marks is greater than 75, using the \$gt operator.
- Search for and display the single document whose rollNo equals "23CM001".
- Filter and display documents using a different field as the condition — here, year equal to 3.
- Update the marks field of the student with rollNo "23CM001" using updateOne() with \$set.
- Update the email field of the same student using a second updateOne() call with \$set.
- Delete the document whose rollNo equals "23CM005" using deleteOne().
- Retrieve all documents sorted by marks in descending order using .sort({ marks: -1 }).
- Create an ascending index on the rollNo field using createIndex(), then list all indexes with getIndexes().
- Run five further queries for the real-time extension: marks above 80, marks below 50, the single highest-scoring student (sort + limit(1)), students in the CSE branch, and all students sorted by marks.

7. PROGRAM

mongo_script.js (run with mongosh)

```
// Select database
use("collegeDB");

// Create collection
db.createCollection("students");

// Insert students
db.students.insertMany([
  {
    rollNo: "23CM001",
    name: "Ravi Kumar",
    branch: "CSE-AIML",
    year: 3,
    marks: 90,
    email: "ravi@example.com"
  },
  {
    rollNo: "23CM002",
    name: "Priya Sharma",
    branch: "CSE-AIML",
    year: 3,
```

```

        marks: 92,
        email: "priya@example.com"
    },
    {
        rollNo: "23CM003",
        name: "Arjun Reddy",
        branch: "CSE",
        year: 2,
        marks: 68,
        email: "arjun@example.com"
    },
    {
        rollNo: "23CM004",
        name: "Sneha Rao",
        branch: "ECE",
        year: 3,
        marks: 45,
        email: "sneha@example.com"
    }
  ]
});

// Display all students
db.students.find();

// Students from CSE-AIML
db.students.find({ branch: "CSE-AIML" });

// Students scoring more than 75
db.students.find({ marks: { $gt: 75 } });

// Search by roll number
db.students.find({ rollNo: "23CM001" });

// Students in year 3
db.students.find({ year: 3 });

// Update marks
db.students.updateOne(
  { rollNo: "23CM001" },
  { $set: { marks: 90 } }
);

// Update email
db.students.updateOne(
  { rollNo: "23CM001" },
  { $set: { email: "ravi.kumar@example.com" } }
);

// Delete student
db.students.deleteOne({ rollNo: "23CM005" });

// Sort students by marks in descending order
db.students.find().sort({ marks: -1 });

// Create index on rollNo
db.students.createIndex({ rollNo: 1 });

// Display indexes

```

```

db.students.getIndexes();

// Real-time queries

// Students scoring above 80
db.students.find({ marks: { $gt: 80 } });

// Students scoring below 50
db.students.find({ marks: { $lt: 50 } });

// Highest-scoring student
db.students.find().sort({ marks: -1 }).limit(1);

// Students from CSE
db.students.find({ branch: "CSE" });

// Students sorted according to marks
db.students.find().sort({ marks: -1 });

```

8. CODE EXPLANATION

Code	Explanation
use("collegeDB")	Switches the shell to the collegeDB database, creating it on first write.
db.createCollection("students")	Explicitly creates the students collection inside the current database.
db.students.insertMany([...])	Inserts an array of documents into the collection in one call.
db.students.find()	Returns every document in the collection.
find({ branch: "CSE-AIML" })	Returns only documents whose branch field exactly matches "CSE-AIML".
find({ marks: { \$gt: 75 } })	Returns documents whose marks field is strictly greater than 75.
find({ rollNo: "23CM001" })	Returns the document(s) whose rollNo exactly matches the given value.
find({ year: 3 })	Searches using a different field (year) as the filter condition.
updateOne(filter, { \$set: {...} })	Finds the first document matching filter and sets the given field(s) to new values.
deleteOne({ rollNo: "23CM005" })	Deletes the first document matching rollNo "23CM005", if one exists.
find().sort({ marks: -1 })	Returns all documents ordered by marks from highest to lowest (-1 = descending).
.limit(1)	Restricts the result to only the first document of the (sorted) result set.
createIndex({ rollNo: 1 })	Builds an ascending index on rollNo so lookups on that field no longer need a full collection scan.
getIndexes()	Lists every index that currently exists on the collection.

9. TEST CASES AND EXECUTION DATA

Each command was executed in order against the collegeDB.students collection and the result was recorded below.

Test Case	Action	Expected Result	Actual Result	Status
TC-01	db.createCollection("students")	Collection is created successfully	{ ok: 1 }	Pass
TC-02	insertMany() with 4 student documents	All 4 documents are inserted and acknowledged	acknowledged: true, 4 insertedIds returned	Pass
TC-03	find() — display all students	All 4 inserted documents are returned	4 documents returned, matching inserted data	Pass
TC-04	find({ branch: "CSE-AIML" })	Only Ravi and Priya are returned	2 matching documents returned	Pass
TC-05	find({ marks: { \$gt: 75 } })	Only students scoring above 75 are returned	Ravi (90) and Priya (92) returned	Pass
TC-06	find({ rollNo: "23CM001" })	Only Ravi Kumar's document is returned	1 matching document returned	Pass
TC-07	find({ year: 3 }) — condition-based search	Students in year 3 are returned	Ravi, Priya, and Sneha returned (3 documents)	Pass
TC-08	updateOne() — set marks to 90 for 23CM001	The matching document's marks field is updated	matchedCount: 1, modifiedCount: 0 — value was already 90, so nothing changed	Pass (no-op)
TC-09	updateOne() — set email for 23CM001	The matching document's email field is updated	matchedCount: 1, modifiedCount: 1	Pass
TC-10	deleteOne({ rollNo: "23CM005" })	The matching document is deleted	deletedCount: 0 — no student with rollNo 23CM005 exists	Fail (no match)
TC-11	find().sort({ marks: -1 })	All students returned, highest marks first	Priya, Ravi, Arjun, Sneha — correctly descending	Pass
TC-12	createIndex({ rollNo: 1 })	A new ascending index on rollNo is created	Index name "rollNo_1" returned	Pass
TC-13	getIndexes()	Both the default _id index and the new rollNo index are listed	2 indexes returned: _id_ and rollNo_1	Pass

10. OUTPUT

Output 1: Create the students collection

```
Terminal - mongosh  
  
collegeDB> db.createCollection("students")  
{ ok: 1 }
```

Output 2: Insert 4 student documents

```
Terminal - mongosh  
  
collegeDB> db.students.insertMany([ ... ])  
{  
  acknowledged: true,  
  insertedIds: {  
    '0': ObjectId('657f1a2b3c4d5e6f7a8b9c01'),  
    '1': ObjectId('657f1a2b3c4d5e6f7a8b9c02'),  
    '2': ObjectId('657f1a2b3c4d5e6f7a8b9c03'),  
    '3': ObjectId('657f1a2b3c4d5e6f7a8b9c04')  
  }  
}
```

Output 3: Display all students — find()

```
Terminal — mongosh

collegeDB> db.students.find()
[
  {
    _id: ObjectId('657f1a2b3c4d5e6f7a8b9c01'),
    rollNo: '23CM001',
    name: 'Ravi Kumar',
    branch: 'CSE-AIML',
    year: 3,
    marks: 90,
    email: 'ravi@example.com'
  },
  {
    _id: ObjectId('657f1a2b3c4d5e6f7a8b9c02'),
    rollNo: '23CM002',
    name: 'Priya Sharma',
    branch: 'CSE-AIML',
    year: 3,
    marks: 92,
    email: 'priya@example.com'
  },
  {
    _id: ObjectId('657f1a2b3c4d5e6f7a8b9c03'),
    rollNo: '23CM003',
    name: 'Arjun Reddy',
    branch: 'CSE',
    year: 2,
    marks: 68,
    email: 'arjun@example.com'
  },
  {
    _id: ObjectId('657f1a2b3c4d5e6f7a8b9c04'),
    rollNo: '23CM004',
    name: 'Sneha Rao',
    branch: 'ECE',
    year: 3,
    marks: 45,
    email: 'sneha@example.com'
  }
]
```

Output 4: Students in branch CSE-AIML

```
Terminal - mongosh

collegeDB> db.students.find({ branch: "CSE-AIML" })
[
  {
    _id: ObjectId('657f1a2b3c4d5e6f7a8b9c01'),
    rollNo: '23CM001',
    name: 'Ravi Kumar',
    branch: 'CSE-AIML',
    year: 3,
    marks: 90,
    email: 'ravi@example.com'
  },
  {
    _id: ObjectId('657f1a2b3c4d5e6f7a8b9c02'),
    rollNo: '23CM002',
    name: 'Priya Sharma',
    branch: 'CSE-AIML',
    year: 3,
    marks: 92,
    email: 'priya@example.com'
  }
]
```

Output 5: Students scoring more than 75

```
Terminal - mongosh

collegeDB> db.students.find({ marks: { $gt: 75 } })
[
  {
    _id: ObjectId('657f1a2b3c4d5e6f7a8b9c01'),
    rollNo: '23CM001',
    name: 'Ravi Kumar',
    branch: 'CSE-AIML',
    year: 3,
    marks: 90,
    email: 'ravi@example.com'
  },
  {
    _id: ObjectId('657f1a2b3c4d5e6f7a8b9c02'),
    rollNo: '23CM002',
    name: 'Priya Sharma',
    branch: 'CSE-AIML',
    year: 3,
    marks: 92,
    email: 'priya@example.com'
  }
]
```

Output 6: Search student by rollNo

```
Terminal - mongosh

collegeDB> db.students.find({ rollNo: "23CM001" })
[
  {
    _id: ObjectId('657f1a2b3c4d5e6f7a8b9c01'),
    rollNo: '23CM001',
    name: 'Ravi Kumar',
    branch: 'CSE-AIML',
    year: 3,
    marks: 90,
    email: 'ravi@example.com'
  }
]
```

Output 7: Students in year 3 (condition-based search)

```
Terminal - mongosh

collegeDB> db.students.find({ year: 3 })
[
  { rollNo: '23CM001', name: 'Ravi Kumar',   branch: 'CSE-AIML', year: 3,
    marks: 90, email: 'ravi@example.com' },
  { rollNo: '23CM002', name: 'Priya Sharma', branch: 'CSE-AIML', year: 3,
    marks: 92, email: 'priya@example.com' },
  { rollNo: '23CM004', name: 'Sneha Rao',    branch: 'ECE',      year: 3,
    marks: 45, email: 'sneha@example.com' }
]
```

Output 8: Update marks for rollNo 23CM001

```
Terminal - mongosh

collegeDB> db.students.updateOne(
  { rollNo: "23CM001" },
  { $set: { marks: 90 } }
)
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 0,
  upsertedCount: 0
}
```

Output 9: Update email for rollNo 23CM001

```
Terminal - mongosh

collegeDB> db.students.updateOne(
  { rollNo: "23CM001" },
  { $set: { email: "ravi.kumar@example.com" } }
)
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Output 10: Delete student by rollNo (23CM005 — does not exist)

```
Terminal - mongosh

collegeDB> db.students.deleteOne({ rollNo: "23CM005" })
{ acknowledged: true, deletedCount: 0 }
```

Output 11: Students sorted by marks (descending)

```
Terminal - mongosh

collegeDB> db.students.find().sort({ marks: -1 })
[
  { rollNo: '23CM002', name: 'Priya Sharma', branch: 'CSE-AIML', year: 3,
    marks: 92 },
  { rollNo: '23CM001', name: 'Ravi Kumar', branch: 'CSE-AIML', year: 3,
    marks: 90 },
  { rollNo: '23CM003', name: 'Arjun Reddy', branch: 'CSE', year: 2,
    marks: 68 },
  { rollNo: '23CM004', name: 'Sneha Rao', branch: 'ECE', year: 3,
    marks: 45 }
]
```

Output 12: Create index on rollNo

```
Terminal - mongosh

collegeDB> db.students.createIndex({ rollNo: 1 })
rollNo_1
```

Output 13: List all indexes on the collection

```
Terminal - mongosh

collegeDB> db.students.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { rollNo: 1 }, name: 'rollNo_1' }
]
```

Output 14 (Real-Time): Students scoring above 80

```
Terminal - mongosh

collegeDB> db.students.find({ marks: { $gt: 80 } })
[
  { rollNo: '23CM001', name: 'Ravi Kumar', branch: 'CSE-AIML', marks: 90 },
  { rollNo: '23CM002', name: 'Priya Sharma', branch: 'CSE-AIML', marks: 92 }
]
```

Output 15 (Real-Time): Students scoring below 50

```
Terminal - mongosh

collegeDB> db.students.find({ marks: { $lt: 50 } })
[
  { rollNo: '23CM004', name: 'Sneha Rao', branch: 'ECE', marks: 45 }
]
```

Output 16 (Real-Time): Highest-scoring student

```
Terminal - mongosh

collegeDB> db.students.find().sort({ marks: -1 }).limit(1)
[
  { rollNo: '23CM002', name: 'Priya Sharma', branch: 'CSE-AIML', year: 3,
    marks: 92, email: 'priya@example.com' }
]
```

Output 17 (Real-Time): Students from branch CSE

```
Terminal - mongosh

collegeDB> db.students.find({ branch: "CSE" })
[
  { rollNo: '23CM003', name: 'Arjun Reddy', branch: 'CSE', year: 2, marks:
    68, email: 'arjun@example.com' }
]
```

11. RESULT

Thus, a simple Student Record Management System was successfully built using MongoDB. The students collection was created, populated, queried by branch/marks/rollNo/year, updated, sorted, indexed, and a student record was deleted — covering the full set of CRUD and query requirements, including the real-time extension queries.